Índice

Write short units of code 3
- **comprarProducto DataAccess** 3
- **getPublishedSales DataAccess** 4

Write simple units of code 6
- **borrarVenta DataAccess** 6
- **Líneas 87-133 en QuerySalesGUI** 7

Duplicate code 9
- **Líneas 42 a 45 ResourceBundle.getBundle("Etiquetas") CreateSaleGUI** 9
- **aceptarOferta con aceptarOfertaIni DataAcess** 10

Keep unit interfaces small 12
- **editSale DataAccess** 12
- **createSale DataAccess** 14

Write short units of code

comprarProducto DataAccess

Código Inicial:

```
362●    public boolean comprarProducto(String usuario, Sale pro) {
363         open();
364         db.getTransaction().begin();
365         try {
366             Comprador comprador = db.find(Comprador.class, usuario);
367             Sale producto = db.find(Sale.class, pro.getSaleNumber());
368             if (comprador != null && producto !=null && producto.isHabilitado() && comprador.getSaldo() >= producto.getPrice()) {
369 
370                 comprador.setSaldo(comprador.getSaldo() - producto.getPrice());
371                 comprador.getHistorialDeCompras().add(producto);
372                 producto.setHabilitado(false);
373 
374 
375                 if (producto.getSeller() != null) {
376                     Seller vendedor = db.find(Seller.class, producto.getSeller().getNombreUsuario());
377                     if (vendedor != null) {
378                         producto.setUsuarioVendedor(vendedor.getNombreUsuario());
379                         vendedor.setSaldo(vendedor.getSaldo() + producto.getPrice());
380                         vendedor.getSales().remove(producto);
381                         db.persist(vendedor);
382                     }
383                 }
384                 db.persist(comprador);
385                 db.persist(producto);
386                 db.getTransaction().commit();
387                 close();
388                 return true;
389             }
390 
391             close();
392             return false;
393         }
394         catch (Exception e){
395             close();
396             return false;
397         }
398     }
```

Código refactorizado:

```
362●    public boolean comprarProducto(String usuario, Sale pro) {
363         open();
364         db.getTransaction().begin();
365         try {
366             Comprador comprador = db.find(Comprador.class, usuario);
367             Sale producto = db.find(Sale.class, pro.getSaleNumber());
368             if (comprador != null && producto !=null && producto.isHabilitado() && comprador.getSaldo() >= producto.getPrice()) {
369                 intercambioProductos(comprador, producto);
370                 db.persist(comprador);
371                 db.persist(producto);
372                 db.getTransaction().commit();
373                 close();
374                 return true;
375             }
376             close();
377             return false;
378         }
379         catch (Exception e){
380             close();
381             return false;
382         }
383     }
384 
385●    private void intercambioProductos(Comprador comprador, Sale producto) {
386         comprador.setSaldo(comprador.getSaldo() - producto.getPrice());
387         comprador.getHistorialDeCompras().add(producto);
388         producto.setHabilitado(false);
389         if (producto.getSeller() != null) {
390             Seller vendedor = db.find(Seller.class, producto.getSeller().getNombreUsuario());
391             if (vendedor != null) {
392                 producto.setUsuarioVendedor(vendedor.getNombreUsuario());
393                 vendedor.setSaldo(vendedor.getSaldo() + producto.getPrice());
394                 vendedor.getSales().remove(producto);
395                 db.persist(vendedor);
396             }
397         }
398     }
```

Descripción del code smell detectado:
Podemos comprobar que, inicialmente, el método comprarProducto tiene más de 15 líneas de código; lo cual dificulta el testeo del método y su entendimiento. Principalmente, lo que más código innecesario crea son los cambios e interacciones con los vendedores y compradores.

Descripción de la refactorización realizada:
Para eliminar este error de “bad smell”, hemos decidido meter toda la transacción principal que ejecuta el método dentro de otro método llamado intercambioProductos().

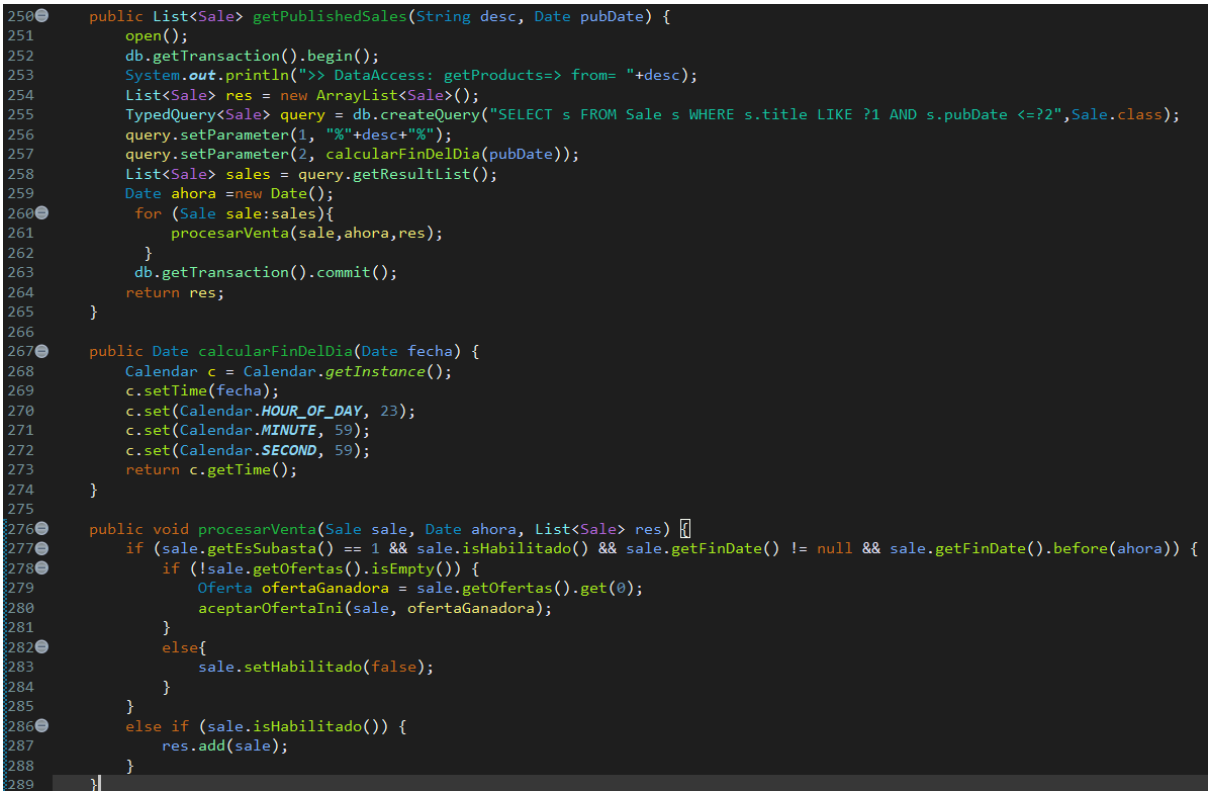
Miembro que ha realizado la refactorización: Joxe Echeveste González

getPublishedSales DataAccess

Código Inicial:

```
250    public List<Sale> getPublishedSales(String desc, Date pubDate) {
251        open();
252        db.getTransaction().begin();
253        Calendar c = Calendar.getInstance();
254        c.setTime(pubDate);
255        c.set(Calendar.HOUR_OF_DAY, 23);
256        c.set(Calendar.MINUTE, 59);
257        c.set(Calendar.SECOND, 59);
258        Date finDelDia = c.getTime();
259        System.out.println(">> DataAccess: getProducts=> from= "+desc);
260        List<Sale> res = new ArrayList<Sale>();
261        TypedQuery<Sale> query = db.createQuery("SELECT s FROM Sale s WHERE s.title LIKE ?1 AND s.pubDate <=?2",Sale.class);
262        query.setParameter(1, "%"+desc+"%");
263        query.setParameter(2, finDelDia);
264        List<Sale> sales = query.getResultList();
265        Date ahora =new Date();
266         for (Sale sale:sales){
267            // nuevo
268                if (sale.getEsSubasta() == 1 && sale.isHabilitado() && sale.getFinDate() != null && sale.getFinDate().before(ahora)) {
269
270                    if (!sale.getOfertas().isEmpty()) {
271                        Oferta ofertaGanadora = sale.getOfertas().get(0);
272                        aceptarOfertaIni(sale, ofertaGanadora);
273                    }
274                    else{
275                        sale.setHabilitado(false);
276                    }
277                }
278
279                else if (sale.isHabilitado()) {
280                    res.add(sale);
281                }
282          }
283         db.getTransaction().commit();
284        return res;
285    }
```

Código refactorizado:



```java
public List<Sale> getPublishedSales(String desc, Date pubDate) {
    open();
    db.getTransaction().begin();
    System.out.println(">> DataAccess: getProducts=> from= "+desc);
    List<Sale> res = new ArrayList<Sale>();
    TypedQuery<Sale> query = db.createQuery("SELECT s FROM Sale s WHERE s.title LIKE ?1 AND s.pubDate <=?2",Sale.class);
    query.setParameter(1, "%"+desc+"%");
    query.setParameter(2, calcularFinDelDia(pubDate));
    List<Sale> sales = query.getResultList();
    Date ahora =new Date();
     for (Sale sale:sales){
         procesarVenta(sale,ahora,res);
      }
     db.getTransaction().commit();
    return res;
}

public Date calcularFinDelDia(Date fecha) {
    Calendar c = Calendar.getInstance();
    c.setTime(fecha);
    c.set(Calendar.HOUR_OF_DAY, 23);
    c.set(Calendar.MINUTE, 59);
    c.set(Calendar.SECOND, 59);
    return c.getTime();
}

public void procesarVenta(Sale sale, Date ahora, List<Sale> res) {
    if (sale.getEsSubasta() == 1 && sale.isHabilitado() && sale.getFinDate() != null && sale.getFinDate().before(ahora)) {
        if (!sale.getOfertas().isEmpty()) {
            Oferta ofertaGanadora = sale.getOfertas().get(0);
            aceptarOfertaIni(sale, ofertaGanadora);
        }
        else{
            sale.setHabilitado(false);
        }
    }
    else if (sale.isHabilitado()) {
        res.add(sale);
    }
}
```

Descripción del code smell detectado:
El método original era demasiado largo, superando el límite de 15 líneas recomendado. El problema principal es que intentaba hacer demasiadas cosas a la vez dentro del mismo bloque de código: calcular las horas de las fechas, buscar las ventas en la base de datos y además cambiar el estado de las subastas.

Descripción de la refactorización realizada:
Para solucionarlo, se ha dividido el código largo en trozos más pequeños creando métodos de apoyo. Por un lado, se han sacado todas las líneas que calculan la fecha y se han puesto en un nuevo método llamado `calcularFinDelDia()`. Por otro lado, la parte que decide qué hacer con cada subasta se ha movido al método `procesarVenta()`.

Miembro que ha realizado la refactorización: Ane Miren Alvarez

Write simple units of code

borrarVenta DataAccess

Código Inicial:

```
655     public void borrarVenta(Sale s) {
656         open();
657         db.getTransaction().begin();
658         Sale saleEnBD = db.find(Sale.class, s.getSaleNumber());
659         if (saleEnBD != null) {
660             if (!saleEnBD.getOfertas().isEmpty()) {
661                 List<Oferta> ofertasADevolver = new ArrayList<>(saleEnBD.getOfertas());
662                 for(Oferta ofertaAct : ofertasADevolver) {
663                     Comprador c = db.find(Comprador.class, ofertaAct.getnUser());
664 
665                     if (c != null) {
666                         float nuevoSaldo = c.getSaldo() + ofertaAct.getPrecio();
667                         c.setSaldo(nuevoSaldo);
668                         c.getOfertasEnCurso().remove(ofertaAct);
669                         db.persist(c);
670                     }
671                     db.remove(ofertaAct);
672                 }
673                 saleEnBD.getOfertas().clear();
674             }
675 
676             if (saleEnBD.getSeller() != null) {
677                 Seller vendedor = db.find(Seller.class, saleEnBD.getSeller().getNombreUsuario());
678                 if (vendedor != null) {
679                     vendedor.getSales().remove(saleEnBD);
680                     db.persist(vendedor);
681                 }
682             }
683 
684             TypedQuery<Comprador> query = db.createQuery("SELECT c FROM Comprador c JOIN c.historialDeCompras h WHERE h = ?1",Comprador.class);
685             query.setParameter(1, saleEnBD);
686 
687             List<Comprador> comprador = query.getResultList();
688 
689             if (!comprador.isEmpty()) {
690                 Comprador cElim = db.find(Comprador.class, comprador.get(0).getNombreUsuario());
691                 cElim.getHistorialDeCompras().remove(saleEnBD);
692                 db.persist(cElim);
693             }
694 
695             db.remove(saleEnBD);
696             db.getTransaction().commit();
697         }
698         close();
699     }
```

Código refactorizado:

```java
public void borrarVenta(Sale s) {
    open();
    db.getTransaction().begin();
    Sale saleEnBD = db.find(Sale.class, s.getSaleNumber());
    if (saleEnBD != null) {
    tieneOfertas(saleEnBD);
    tieneSeller(saleEnBD);
    tieneComprador(saleEnBD);

        db.remove(saleEnBD);
        db.getTransaction().commit();
    }
    close();
}

public void tieneOfertas(Sale saleEnBD) {
    if (!saleEnBD.getOfertas().isEmpty()) {
        List<Oferta> ofertasADevolver = new ArrayList<>(saleEnBD.getOfertas());
        for(Oferta ofertaAct : ofertasADevolver) {
            Comprador c = db.find(Comprador.class, ofertaAct.getnUser());

            if (c != null) {
                float nuevoSaldo = c.getSaldo() + ofertaAct.getPrecio();
                c.setSaldo(nuevoSaldo);
                c.getOfertasEnCurso().remove(ofertaAct);
                db.persist(c);
            }
            db.remove(ofertaAct);
        }
        saleEnBD.getOfertas().clear();
    }
}

public void tieneSeller(Sale saleEnBD) {
    if (saleEnBD.getSeller() != null) {
        Seller vendedor = db.find(Seller.class, saleEnBD.getSeller().getNombreUsuario());
        if (vendedor != null) {
            vendedor.getSales().remove(saleEnBD);
            db.persist(vendedor);
        }
    }
}

public void tieneComprador(Sale saleEnBD) {
    TypedQuery<Comprador> query = db.createQuery("SELECT c FROM Comprador c JOIN c.historialDeCompras h WHERE h = ?1",Comprador.class);
    query.setParameter(1, saleEnBD);

    List<Comprador> comprador = query.getResultList();

    if (!comprador.isEmpty()) {
        Comprador cElim = db.find(Comprador.class, comprador.get(0).getNombreUsuario());
        cElim.getHistorialDeCompras().remove(saleEnBD);
        db.persist(cElim);
    }
}
```

Descripción del code smell detectado:
La función inicial presentaba una cantidad excesiva de estructuras condicionales (`if`) y bucles (`for`) iterativos entrelazados. Esta acumulación elevaba el número total de bifurcaciones, sobrepasando la restricción de 4 puntos de ramificación establecida.

Descripción de la refactorización realizada:
Con el fin de mitigar dicha complejidad, la lógica interna se ha extraído mediante tres métodos de apoyo independientes: `tieneOfertas()`, `tieneSeller()` y `tieneComprador()`. Con estas modificaciones, la función principal `borrarVenta()` únicamente verifica la existencia del registro y delega la ejecución en los métodos auxiliares. Así, ninguna de las funciones resultantes excede el tope de 4 puntos de ramificación, garantizando un código más accesible, sencillo de modificar y fácil de probar.

Miembro que ha realizado la refactorización: Ane Miren Alvarez

Líneas 87-133 en QuerySalesGUI

Código Inicial:

```java
        jButtonSearch.addActionListener(new ActionListener() {
           public void actionPerformed(ActionEvent e) {
               try {
                   tableModelProducts.setDataVector(null, columnNamesProducts);
                   tableModelProducts.setColumnCount(5); // another column added to allocate product object

                   BLFacade facade = MainGUI.getBusinessLogic();
                   Date today = UtilDate.trim(new Date());

                   List<Sale> sales=facade.getPublishedSales(jTextFieldSearch.getText(),today);

                   Usuario userActual = facade.buscarPorUser(usuario);

                   if (sales.isEmpty() ) jLabelProducts.setText("No hay ventas disponibles");
                   else jLabelProducts.setText(ResourceBundle.getBundle("Etiquetas").getString("QuerySalesGUI.Products"));
                   for (Sale sale:sales){
                       if (sale.isHabilitado()) {
                           if (!(userActual instanceof Friendly && sale.getEsSubasta() == 1)) {
                               Vector<Object> row = new Vector<Object>();
                               row.add(sale.getTitle());
                               row.add(sale.getPrice());
                               row.add(new SimpleDateFormat("dd-MM-yyyy").format(sale.getPublicationDate()));
                               int tVenta = sale.getEsSubasta();
                               if(tVenta==0) {
                                   row.add("Venta");
                               }
                               else {
                                   row.add("Subasta");
                               }
                               row.add(sale); // product object added in order to obtain it with tableModelProducts.getValueAt(i,2)
                               tableModelProducts.addRow(row);
                           }
                       }
                   }
               } catch (Exception e1) {

                   e1.printStackTrace();
               }
               tableProducts.getColumnModel().getColumn(0).setPreferredWidth(100);
               tableProducts.getColumnModel().getColumn(1).setPreferredWidth(110);
               tableProducts.getColumnModel().getColumn(2).setPreferredWidth(70);
               tableProducts.getColumnModel().getColumn(3).setPreferredWidth(80);

               tableProducts.getColumnModel().removeColumn(tableProducts.getColumnModel().getColumn(4)); // not shown in JTable

           }
        });
```

Código refactorizado:

```
        jButtonSearch.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try {
                    tableModelProducts.setDataVector(null, columnNamesProducts);
                    tableModelProducts.setColumnCount(5); // another column added to allocate product object

                    BLFacade facade = MainGUI.getBusinessLogic();
                    Date today = UtilDate.trim(new Date());

                    List<Sale> sales=facade.getPublishedSales(jTextFieldSearch.getText(),today);

                    Usuario userActual = facade.buscarPorUser(usuario);

                    if (sales.isEmpty() ) jLabelProducts.setText("No hay ventas disponibles");
                    else jLabelProducts.setText(ResourceBundle.getBundle("Etiquetas").getString("QuerySalesGUI.Products"));
                    for (Sale sale:sales){
                        anadirSale(userActual, sale);
                    }
                } catch (Exception e1) {

                    e1.printStackTrace();
                }
                tableProducts.getColumnModel().getColumn(0).setPreferredWidth(100);
                tableProducts.getColumnModel().getColumn(1).setPreferredWidth(110);
                tableProducts.getColumnModel().getColumn(2).setPreferredWidth(70);
                tableProducts.getColumnModel().getColumn(3).setPreferredWidth(80);

                tableProducts.getColumnModel().removeColumn(tableProducts.getColumnModel().getColumn(4)); // not shown in JTable

            }

            private void anadirSale(Usuario userActual, Sale sale) {
                if (sale.isHabilitado()) {
                    if (!(userActual instanceof Friendly && sale.getEsSubasta() == 1)) {
                        Vector<Object> row = new Vector<Object>();
                        row.add(sale.getTitle());
                        row.add(sale.getPrice());
                        row.add(new SimpleDateFormat("dd-MM-yyyy").format(sale.getPublicationDate()));
                        int tVenta = sale.getEsSubasta();
                        if(tVenta==0) {
                            row.add("Venta");
                        }
                        else {
                            row.add("Subasta");
                        }
                        row.add(sale); // product object added in order to obtain it with tableModelProducts.getValueAt(i,2)
                        tableModelProducts.addRow(row);
                    }
                }
            }
        });
```

Descripción del code smell detectado:
Hemos comprobado que en este eventListener para jButtonSearch se encuentran más de 4 estructuras condicionales, sobre todo IFs. Esto genera una complejidad de método innecesaria al poder simplificarlo dividiendo el método en unidades.

Descripción de la refactorización realizada:
Para poder simplificar este problema, creamos un método llamado anadirSale que se centre en añadir las ventas a la tabla si es que se encuentra disponible; y lo añadimos en el for.

Miembro que ha realizado la refactorización: Joxe Echeveste González

Duplicate code

Líneas 42 a 45 ResourceBundle.getBundle("Etiquetas") CreateSaleGUI

Código Inicial:

```
42      private JLabel jLabelTitle = new JLabel(ResourceBundle.getBundle("Etiquetas").getString("CreateSaleGUI.Title"));
43      private JLabel jLabelDescription = new JLabel(ResourceBundle.getBundle("Etiquetas").getString("CreateSaleGUI.Description"));
44      private JLabel jLabelProductStatus = new JLabel(ResourceBundle.getBundle("Etiquetas").getString("CreateSaleGUI.Status"));
45      private JLabel jLabelPrice = new JLabel(ResourceBundle.getBundle("Etiquetas").getString("CreateSaleGUI.Price"));
46      private JTextField jTextFieldPrice = new JTextField();
```

Código refactorizado:

```
42      private JLabel jLabelTitle = crearLabel("CreateSaleGUI.Title");
43      private JLabel jLabelDescription = crearLabel("CreateSaleGUI.Description");
44      private JLabel jLabelProductStatus = crearLabel("CreateSaleGUI.Status");
45      private JLabel jLabelPrice = crearLabel("CreateSaleGUI.Price");
```

```
66      public JLabel crearLabel(String s) {
67          return new JLabel(ResourceBundle.getBundle("Etiquetas").getString(s));
68      }
```

Descripción del code smell detectado:
Al construir los elementos de la interfaz, se duplicaba de forma continuada la llamada `ResourceBundle.getBundle("Etiquetas").getString(...)` al asignar el texto de cada `JLabel`.

Descripción de la refactorización realizada:
Para solventar esta duplicidad, la lógica repetida se ha extraído a un método auxiliar denominado `crearLabel()`. Dicho procedimiento recibe como parámetro la clave correspondiente del texto a recuperar, consulta la cadena en el archivo y retorna la instancia de `JLabel` completamente construida. Gracias al uso del nuevo método se erradica el código duplicado, simplificando sensiblemente la declaración de los componentes gráficos

Miembro que ha realizado la refactorización: Ane Miren Alvarez

aceptarOferta con aceptarOfertaIni DataAcess

Código Inicial:

```
602     public void aceptarOferta(Sale s, Oferta o) {
603         open();
604         db.getTransaction().begin();
605         Sale sA = db.find(Sale.class,s.getSaleNumber());
606         Oferta dOf = db.find(Oferta.class, o.getOfertaNumber());
607         Seller sel= db.find(Seller.class, s.getSeller());
608         Comprador com = db.find(Comprador.class, dOf.getnUser());
609         com.getOfertasEnCurso().remove(dOf);
610         float nuevoSaldo = sel.getSaldo() + dOf.getPrecio();
611         sel.setSaldo(nuevoSaldo);
612         sA.getOfertas().remove(dOf);
613         sA.setPrice(dOf.getPrecio());
614         com.getHistorialDeCompras().add(sA);
615         sel.getSales().remove(sA);
616         sA.setHabilitado(false);
617         db.remove(dOf);
618         db.getTransaction().commit();
619         close();
620     }
621
622     public void aceptarOfertaIni(Sale s, Oferta o) {
623         Sale sA = db.find(Sale.class,s.getSaleNumber());
624         Oferta dOf = db.find(Oferta.class, o.getOfertaNumber());
625         Seller sel= db.find(Seller.class, s.getSeller());
626         Comprador com = db.find(Comprador.class, dOf.getnUser());
627         com.getOfertasEnCurso().remove(dOf);
628         float nuevoSaldo = sel.getSaldo() + dOf.getPrecio();
629         sel.setSaldo(nuevoSaldo);
630         sA.getOfertas().remove(dOf);
631         sA.setPrice(dOf.getPrecio());
632         com.getHistorialDeCompras().add(sA);
633         sel.getSales().remove(sA);
634         sA.setHabilitado(false);
635         db.remove(dOf);
636     }
```

Código refactorizado:



```
602     public void aceptarOferta(Sale s, Oferta o) {
603         open();
604         db.getTransaction().begin();
605         aceptaOfertas(s, o);
606         db.getTransaction().commit();
607         close();
608     }
609
610     private void aceptaOfertas(Sale s, Oferta o) {
611         Sale sA = db.find(Sale.class,s.getSaleNumber());
612         Oferta dOf = db.find(Oferta.class, o.getOfertaNumber());
613         Seller sel= db.find(Seller.class, s.getSeller());
614         Comprador com = db.find(Comprador.class, dOf.getnUser());
615         com.getOfertasEnCurso().remove(dOf);
616         float nuevoSaldo = sel.getSaldo() + dOf.getPrecio();
617         sel.setSaldo(nuevoSaldo);
618         sA.getOfertas().remove(dOf);
619         sA.setPrice(dOf.getPrecio());
620         com.getHistorialDeCompras().add(sA);
621         sel.getSales().remove(sA);
622         sA.setHabilitado(false);
623         db.remove(dOf);
624     }
625
626     public void aceptarOfertaIni(Sale s, Oferta o) {
627         aceptaOfertas(s, o);
628     }
```

Descripción del code smell detectado:
Los métodos aceptarOferta y aceptarOfertaIni ejecutan prácticamente el mismo código; lo cual podríamos resumir en un único método para no tener código repetido.

Descripción de la refactorización realizada:
Para solucionar este problema, simplemente hemos cogido todo el cogido repetido y metido dentro de un código llamado aceptaOfertas(); y hemos añadido la llamada al método en ambas funciones.

Miembro que ha realizado la refactorización: Joxe Echeveste González

Keep unit interfaces small

editSale DataAccess

Código Inicial:

```
441    public void editSale(Sale sale, String text, int numStatus, float price, Date trim, boolean hab) {
442        open();
443        db.getTransaction().begin();
444        //db.persist(sale);
445        Sale s = db.find(Sale.class, sale.getSaleNumber());
446        //Sale s = sale.getSeller().getSales().get(sale.getSeller().getSales().indexOf(sale));
447        s.setDescription(text);
448        s.setStatus(numStatus);
449        s.setPrice(price);
450        s.setPublicationDate(trim);
451        if (hab) {
452            Calendar c = Calendar.getInstance();
453            c.setTime(s.getPublicationDate());
454            //c.add(Calendar.DAY_OF_MONTH, 7);
455            c.add(Calendar.MINUTE, 2);
456            s.setFinDate(c.getTime());
457        }
458        s.setHabilitado(hab);
459        db.persist(s);
460        //db.persist(sale.getSeller());
461        db.getTransaction().commit();
462        close();
463    }
```

Código refactorizado:

```
441    public void editSale(Sale sale, EditSaleParameter parameterObject) {
442        open();
443        db.getTransaction().begin();
444        //db.persist(sale);
445        Sale s = db.find(Sale.class, sale.getSaleNumber());
446        //Sale s = sale.getSeller().getSales().get(sale.getSeller().getSales().indexOf(sale));
447        s.setDescription(parameterObject.getText());
448        s.setStatus(parameterObject.getNumStatus());
449        s.setPrice(parameterObject.getPrice());
450        s.setPublicationDate(parameterObject.getTrim());
451        if (parameterObject.isHab()) {
452            Calendar c = Calendar.getInstance();
453            c.setTime(s.getPublicationDate());
454            //c.add(Calendar.DAY_OF_MONTH, 7);
455            c.add(Calendar.MINUTE, 2);
456            s.setFinDate(c.getTime());
457        }
458        s.setHabilitado(parameterObject.isHab());
459        db.persist(s);
460        //db.persist(sale.getSeller());
461        db.getTransaction().commit();
462        close();
463    }
```

EditSaleParameter:

```java
public class EditSaleParameter {
    private String text;
    private int numStatus;
    private float price;
    private Date trim;
    private boolean hab;

    public EditSaleParameter(String text, int numStatus, float price, Date trim, boolean hab) {
        this.text = text;
        this.numStatus = numStatus;
        this.price = price;
        this.trim = trim;
        this.hab = hab;
    }

    public String getText() {
        return text;
    }

    public int getNumStatus() {
        return numStatus;
    }

    public float getPrice() {
        return price;
    }

    public Date getTrim() {
        return trim;
    }

    public boolean isHab() {
        return hab;
    }
}
```

BLFacadeImplementation:

```java
    public void editSale(Sale sale, String text, int numStatus, float price, Date trim, boolean hab) {
        dbManager.editSale(sale, new EditSaleParameter(text, numStatus, price, trim, hab));
    }
```

Descripción del code smell detectado:
La función `editSale()` requería 6 argumentos para llevar a cabo la actualización. Al rebasar la norma recomendada de un máximo de 4 argumentos, la lectura resulta difusa y se incrementa notablemente la posibilidad de cometer fallos al invocar la función.

Descripción de la refactorización realizada:
Para solventar esta deficiencia, los múltiples argumentos se han encapsulado dentro de un objeto contenedor denominado `EditSaleParameter`. Con esta modificación, la firma del procedimiento pasa a recibir únicamente 2 parámetros: la venta en cuestión y la clase estructurada. Adicionalmente, se ha adaptado la invocación en `BLFacadeImplementation` para empaquetar adecuadamente la información con carácter previo al envío, obteniendo una solución considerablemente más clara, mantenible y protegida de errores.

Miembro que ha realizado la refactorización: Ane Miren Alvarez

createSale DataAccess

Código Inicial:

```
    public Sale createSale(String title, String description, int status, float price,  Date pubDate, String usuario, File file, int tVenta) throws  FileNotUploadedException, MustBeLatevThanTodayException, SaleAlreadyExistException {


        LOGGER.info(">> DataAccess: createProduct=> title=" + title + " usuario=" + usuario);
        try {


            if(pubDate.before(UtilDate.trim(new Date()))) {
                throw new MustBeLaterThanTodayException(ResourceBundle.getBundle("Etiquetas").getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
            }
            if (file==null)
                throw new FileNotUploadedException(ResourceBundle.getBundle("Etiquetas").getString("DataAccess.ErrorFileNotUploadedException"));

            db.getTransaction().begin();

            Seller seller = db.find(Seller.class, usuario);
            if (seller.doesSaleExist(title)) {
                db.getTransaction().commit();
                throw new SaleAlreadyExistException(ResourceBundle.getBundle("Etiquetas").getString("DataAccess.SaleAlreadyExist"));
            }




            Sale sale = seller.addSale(title, description, status, price, pubDate, file, tVenta);
            //next instruction can be obviated

            db.persist(seller);
            db.getTransaction().commit();
             System.out.println("sale stored "+sale+ " "+seller);

            return sale;
        } catch (NullPointerException e) {
            e.printStackTrace();
            // TODO Auto-generated catch block
            db.getTransaction().commit();
            return null;
        }


    }
```

Código refactorizado:

```
    public Sale createSale(String title, String description, int status, CreateSaleParameter infoAdic) throws  FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException
        float price = infoAdic.getPrice();
        String usuario = infoAdic.getUsuario();
        Date pubDate = infoAdic.getPubDate();
        File file = infoAdic.getFile();
        int tVenta = infoAdic.getTVenta();

        LOGGER.info(">> DataAccess: createProduct=> title=" + title + " usuario=" + usuario);
        try {

            if(pubDate.before(UtilDate.trim(new Date()))) {
                throw new MustBeLaterThanTodayException(ResourceBundle.getBundle("Etiquetas").getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
            }
            if (file==null)
                throw new FileNotUploadedException(ResourceBundle.getBundle("Etiquetas").getString("DataAccess.ErrorFileNotUploadedException"));

            db.getTransaction().begin();

            Seller seller = db.find(Seller.class, usuario);
            if (seller.doesSaleExist(title)) {
                db.getTransaction().commit();
                throw new SaleAlreadyExistException(ResourceBundle.getBundle("Etiquetas").getString("DataAccess.SaleAlreadyExist"));
            }




            Sale sale = seller.addSale(title, description, status, price, pubDate, file, tVenta);
            //next instruction can be obviated

            db.persist(seller);
            db.getTransaction().commit();
             System.out.println("sale stored "+sale+ " "+seller);

            return sale;
        } catch (NullPointerException e) {
            e.printStackTrace();
            // TODO Auto-generated catch block
            db.getTransaction().commit();
            return null;
        }


    }
```

CreateSaleParameter:

```java
package dataAccess;

import java.io.File;
import java.util.Date;

public class CreateSaleParameter {
    private float price;
    private Date pubDate;
    private String usuario;
    private File file;
    private int tVenta;

    public CreateSaleParameter( float price, Date pubDate, String usuario, File file, int tVenta) {
        this.price = price;
        this.pubDate = pubDate;
        this.usuario = usuario;
        this.file = file;
        this.tVenta = tVenta;
    }

    public float getPrice() {
        return price;
    }

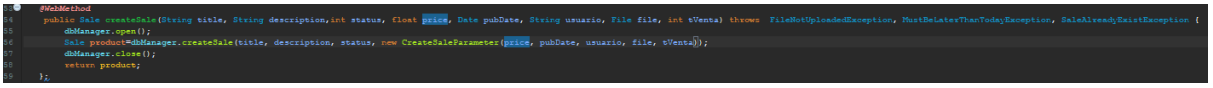
    public Date getPubDate() {
        return pubDate;
    }

    public String getUsuario() {
        return usuario;
    }

    public File getFile() {
        return file;
    }

    public int getTVenta() {
        return tVenta;
    }
}
```

BLFacadeImplementation:



Descripción del code smell detectado:
Como podemos ver en la cabecera del método createSale, el método recibe 8 parámetros en vez de los 4 recomendados; lo cual puede llegar a causar confusiones al intentar llamar al método.

Descripción de la refactorización realizada:
Para solucionar el problema del número elevado de parámetros, hemos recogido los últimos 5 parámetros del método y hemos creado una clase CreateClassParameter la cual se encarga de guardar esos datos como información adicional; a la que después el método accede antes de hacer nada. Como al metodo tambien se le llama en BLFacadeImplementation, hemos actualizado la llamada en el mismo (Como indicado en la imagen)

Miembro que ha realizado la refactorización: Joxe Echeveste González